

Project ID :

Y4 Research Project Group (25-26J-477)

1. Topic (12 words max)

A Human-Like Agent for Screen-Aware Task Automation

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	
Year	

5. Brief description of the research problem including references (200 – 500 words max) – references not included in word count.

In software testing, automation tools have traditionally depended on access to internal system resources like view hierarchies, accessibility APIs, or screen buffers. While effective in modern applications, these techniques are unsuitable for many real-world scenarios - such as legacy systems, or restricted environments - where installing software or accessing internals is not allowed.

Recent research has introduced Vision-Language Models (VLMs) and Large Language Models (LLMs) for GUI automation. These systems visually interpret screens and execute actions from natural language commands, simulating human behavior. For example, **ScreenAgent** [1] implements a perception-planning-acting loop using LLM reasoning, while **SeeClick** [2], **VisionTasker** [3], and **WinClick** [4] enable GUI control via visual grounding and image-based interaction. However, most such solutions assume the automation runs *inside* the target system or has access to screenshots or UI layers - creating a barrier for black-box or locked-down systems.

Some industrial efforts like **Zilogic** [8] and **CERT Polska** [9] have explored USB-based Human Interface Device (HID) emulation to simulate human inputs externally. While effective at hardware-level input injection, these approaches are typically script-driven and lack intelligent task planning or adaptability.

To address these limitations, this research proposes an **external, vision-based testing agent** that integrates four key components:

- **Visual Perception** using a camera and computer vision (OpenCV + OCR) to interpret screen content in real time.
- **LLM Reasoning & Prompt Engineering**, where a large language model translates user instructions into adaptive action plans through carefully crafted prompts - enabling flexible automation even for high-level tasks.
- **Agentic AI Logic**, which orchestrates the overall system using a human-like loop of perception, reasoning, feedback evaluation, and re-planning - crucial for handling exploratory testing flows and dynamic screen changes.
- **ESP32-based HID Emulation with Feedback**, which injects mouse/keyboard inputs via USB and also returns execution status back to the reasoning engine, enabling retries or adjustments when actions fail.

Additionally, the system integrates with **JIRA via API**, enabling automated bug logging, test result tracking, and report generation - making it suitable for real-world QA workflows and continuous integration.

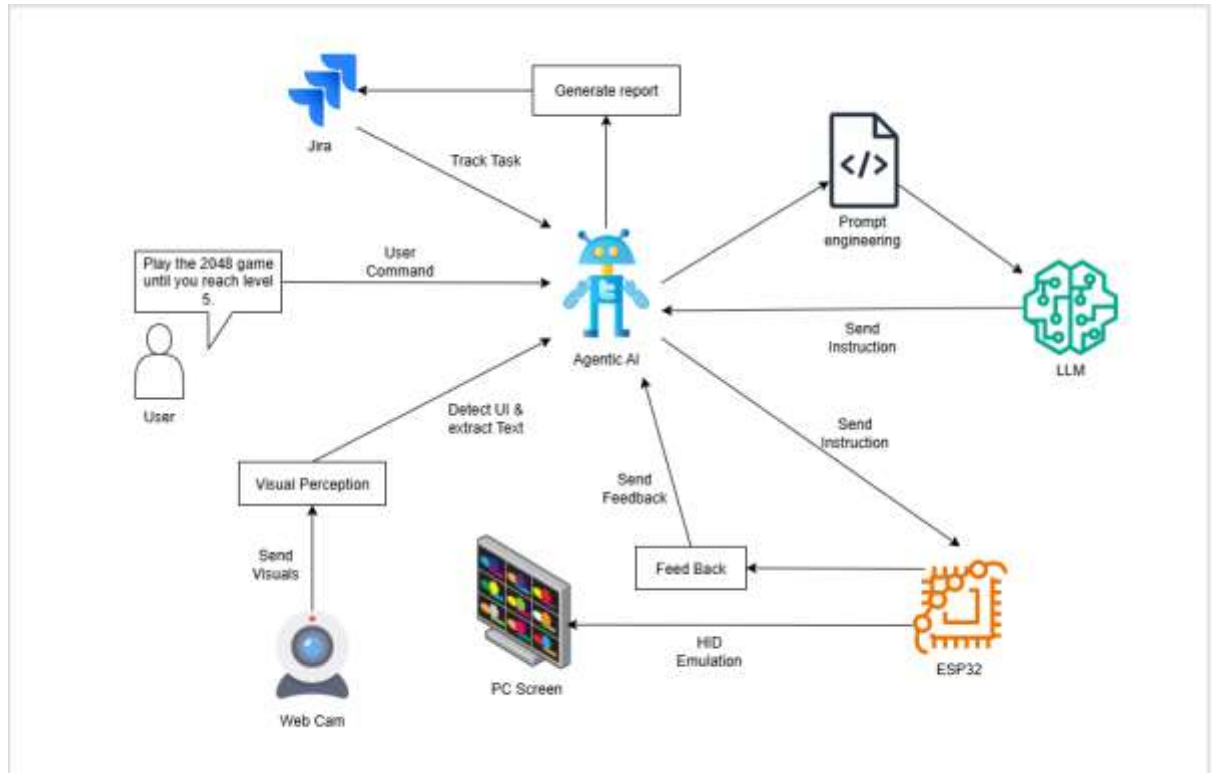
This fully external agent operates independently of the host machine, requiring no installation or internal access. It supports **black-box, cross-platform**, and **undetectable** testing, while mimicking human testers through real-time adaptation and intelligent control.

By combining hardware-level input, vision-based perception, LLM-driven reasoning, and task traceability, the proposed solution fills a significant gap in the current landscape of UI automation and testing tools.

Key Research Papers and Projects

- [1] R. Niu *et al.*, "ScreenAgent: A Vision Language Model-driven Computer Control Agent," Feb. 2024, doi: 10.24963/ijcai.2024/711.
- [2] K. Cheng *et al.*, "SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents," Feb. 2024.
- [3] Y. Song, Y. Bian, Y. Tang, G. Ma, and Z. Cai, "VisionTasker: Mobile Task Automation Using Vision Based UI Understanding and LLM Task Planning," Jul. 2024, doi: 10.1145/3654777.3676386.
- [4] Z. Hui, Y. Li, D. Zhao, T. Chen, C. Banbury, and K. Koishida, "WinClick: GUI Grounding with Multimodal Large Language Models," Jan. 2025.
- [5] R. Anam, M. Rahman, M. O. Haque, and Md. S. Islam, "Computer Vision based Automation System for Detecting Objects," *International Journal of Intelligent Systems and Applications*, vol. 7, no. 12, pp. 68–74, Nov. 2015, doi: 10.5815/ijisa.2015.12.07.
- [6] Y. Wang, H. Zhang, J. Tian, and Y. Tang, "Ponder & Press: Advancing Visual GUI Agent towards General Computer Control," Dec. 2024.
- [7] Y. Xu *et al.*, "Aguvis: Unified Pure Vision Agents for Autonomous GUI Interaction," May 2025.
- [8] Jacobpaul.p, Bineesh.pc, and Vijaykumar, "Case Study: Touch Input Automation," Jan. 2023.
- [9] Jan Gruber, "HID Simulation for DRAKVUF," Aug. 2021.
- [10] piraija, "USB HID & Run Research," Nov. 2023.

6. Brief description of the nature of the solution including a conceptual diagram (250 words max)



Nature of the Solution

This solution is an Agentic AI-powered test automation system designed to replicate human interaction with computer interfaces from outside the target machine. Instead of relying on internal software access or APIs, the system visually observes the computer screen using an external camera (such as a webcam) and performs input actions such as mouse clicks, typing, and scrolling through HID emulation. This approach enables seamless end-to-end test automation for black-box, legacy, or restricted environments where traditional tools are not feasible.

The core of the system functions as an autonomous agent that perceives, reasons, and acts in a closed feedback loop. The process begins when a user provides a natural-language test instruction. Through prompt engineering, these instructions are optimized and structured for the language model to interpret accurately. A language model then interprets the prompt, breaks it down into actionable steps, and leverages a dedicated Agentic AI planning module to organize and manage the sequence of UI interactions based on evolving screen states.

The agent captures real-time images of the screen and applies computer vision techniques (like OCR and object detection) to identify and understand UI elements and their changes. Physical actions are executed by sending precise mouse and keyboard events to the target machine, closely mimicking a real user. After each action, the system re-captures the screen to verify outcomes and adapt its strategy as needed, ensuring robust and adaptive test execution.

In addition, the system integrates with Jira to automatically log each test case, action taken, and observed result, providing structured reporting and traceability aligned with modern development workflows.

By combining perception, structured reasoning, and physical execution, this solution offers a universal, human-like test automation approach that can generalize across diverse interfaces, adapt dynamically to changes, and operate without requiring internal system access.

7. Brief description of specialized domain expertise, knowledge, and data requirements (300 words max)

This research focuses on developing a hybrid autonomous software test agent that would mimic human actions by visually monitoring a computer screen and communicating with programs through physical input like a mouse or keyboard. The system is designed with three main components: a webcam (used as the camera), a host computer (used for processing images and decision-making), and a microcontroller (ESP32) to simulate human inputs.

The subject-matter expertise required encompasses several fields of expertise. First, computer vision expertise is required to interpret screen shots, recognize UI elements, and extract useful objects or text by applying OCR engines like PaddleOCR or Tesseract. Second, AI reasoning with a big language model (LLM) like LLaMA or OpenAI's GPT is required to make sense of the visual state and decide on the next action to perform. This is further supported by a dedicated Agentic AI component responsible for planning and managing reasoning flows, enabling multi-step decision-making based on state transitions. Third, embedded systems knowledge is necessary to program the ESP32 microcontroller, which is used as a human input emulator by feeding keyboard and mouse simulation signals to the target system.

Additionally, prompt engineering is employed to refine and structure user instructions into context-aware inputs that maximize the LLM's effectiveness in test scenarios. For images, the system requires sample screenshots or live screen video streams of the apps or games to test. These screens are used to train or fine-tune the visual processing modules and to verify the detection accuracy and interaction. A range of test case scenarios or task prompts (e.g., "jump over obstacle" or "click start") also help the LLM learn to generalize actions from visual states.

The overall system is a new, hands-off, and realistic test approach that is non-intrusive to the target software, thus its enormous worth for black-box testing, game testing, and accessibility research.

8. Objectives and Novelty

Main Objective To develop an external, camera-driven intelligent agent capable of performing screen-aware test automation through vision-based perception, natural language instruction understanding, and HID-based interaction.			
Member Name with Registration No	Sub Objective	Tasks	Novelty
Annesiyani Srikanthan - IT22125248	Agentic AI Logic	This task involves building the core decision-making engine that controls the system's intelligent behavior. The Agentic AI Logic interprets LLM instructions, uses visual feedback to guide actions, and follows the control loop (Perceive → Plan → Act → Evaluate). It manages test flow, handles retries, and maintains test state (goals, progress, results). A key feature is adaptive exploratory testing - when failures or unexpected results occur, the agent tries alternative paths, mimicking human testers. It also integrates with Jira to auto-log test outcomes, errors, and progress via API. Acting as the internal orchestrator, it coordinates between visual input, prompt plans, and HID actions while producing reports, logs, and test status.	This component implements human-like autonomous reasoning within a screen-aware environment using external perception. Unlike fixed scripts(test cases) or rule-based systems, it dynamically adapts to real-time feedback, enabling exploratory black-box testing. The agent intelligently handles failures by trying alternative paths, similar to human testers, and integrates with Jira to automatically log test results and issues, making the entire testing process adaptive, traceable, and automated.

Hemapriya H. A. N. S - IT22099518	Visual Perception (PC)	This task involves capturing live video of the screen using a webcam and processing each frame with OpenCV to enhance clarity and isolate key interface regions. Vision techniques are applied to detect UI elements such as buttons, fields, and text, while OCR tools like EasyOCR or Tesseract extract readable information from the screen in real time. After each user action, updated frames are compared with previous ones to identify visual changes and verify outcomes. The extracted visual data is structured and passed to the Agentic AI module, enabling it to perceive the current screen state and make informed decisions during task execution. This vision-based module supports black-box automation without requiring internal access to the target application.	Enables screen understanding through an external camera without system-level access. Unlike traditional tools that rely on internal APIs or screen captures, this module uses real-time computer vision and OCR to perceive and track UI changes. By directly feeding this data to the Agentic AI logic, it enables intelligent, adaptive test automation.
H.I.G. Amith Hasintha - IT22167378	HID Actuation (ESP32)	This task involves designing and implementing the HID emulation module using an ESP32-S3 microcontroller. This component acts as the physical actuation layer of the autonomous testing system by emulating keyboard and mouse inputs on the target computer. The ESP32 is programmed to function as a USB Human Interface Device (HID), receiving structured action commands - such as key presses (e.g., "KEY_SPACE") and mouse clicks (e.g., "MOUSE_LEFT_CLICK") - from the Agentic AI logic via Serial communication. Upon receiving a command, the ESP32 parses and executes the corresponding HID behavior in real time, accurately simulating human interaction. To support reliable testing, the module also provides execution feedback back to the Agentic AI, confirming whether a command was received, executed, or failed. This feedback loop allows the system to re-attempt actions if needed and helps maintain synchronization with the visual feedback from the camera. The implementation ensures precise timing, reliable command interpretation, feedback reporting, and consistent input execution.	This ESP32-based HID emulator enables hardware-level input automation driven by Agentic AI logic. Unlike traditional tools that rely on OS-level access or installed software, this module injects real-time keyboard and mouse actions directly via USB, ensuring compatibility and operating in a way that remains undetected by the system. By supporting bidirectional communication, it not only executes input commands but also provides real-time feedback on execution status, enhancing the system's ability to adapt and retry like a human tester.

E.K.K. Tharushi Nethmini Edirisinghe - IT22083296	LLM & Prompt Engineering Module	The tasks involve accepting natural language instructions (e.g., “Log in and check balance”) and using an LLM (such as GPT or LLaMA), supported by prompt engineering techniques, to convert them into a sequence of step-by-step actions (e.g., “Find login field → Type username → Click submit”). This process also fuses visual perception data from the previous module to determine the exact location of each target element on the screen. Based on this, the system generates a structured action plan specifying what action to take and where (e.g., “Click on button labeled ‘Login’ at (x,y)”). The LLM reasoning engine works in close coordination with the Agentic AI logic, which adapts plans in real time if screen elements change or are not detected as expected, ensuring flexible and intelligent task execution.	Our core novelty lies in using LLM-based task planning combined with prompt engineering to automate UI testing from natural language instructions, entirely through external vision and agentic reasoning. This creates a human-like testing agent that adapts in real time without requiring system-level access.
---	--	--	---

9. Individual component description of how it is complied with the specialization.

Member Name with Registration No	Description
Annesiyani Srikanthan - IT22125248	Design and implement the Agentic AI core logic to control testing flows. Monitor test outcomes and handle retries, fallbacks, and adaptive exploratory testing. Manage the feedback loop by re-capturing and verifying screen states. Generate structured test reports, logs, and automatically integrate results with Jira for issue tracking. Coordinate all modules dynamically to ensure smooth task execution.
Hemapriya H. A. N. S - IT22099518	Capture live screen video using a camera and process frames with OpenCV and OCR tools to detect UI elements like buttons and fields. Identify visual changes after each action and provide structured perception data to the Agentic AI module to enable intelligent decision-making. Support black-box testing without system-level access.
H.I.G. Amith Hasintha - IT22167378	Develop the ESP32 HID controller to emulate keyboard and mouse inputs. Receive precise, structured commands from the Agentic AI logic and execute them in real-time to simulate human interactions. Ensure accurate synchronization between input actions and feedback for seamless test automation.
E.K.K. Tharushi Nethmini Edirisinghe - IT22083296	Use prompt engineering and LLMs (GPT or LLaMA) to convert natural language test instructions into detailed, step-by-step action plans. Fuse visual perception data to determine exact UI element locations. Work closely with Agentic AI logic to dynamically adjust plans in response to screen changes and failures, ensuring flexible and adaptive task execution.

10. Supervisor details

	Title	First Name	Last Name	Signature
Supervisor	Mr.	Vishan	Jayasinghearachchi	Vishan
Co-Supervisor	Ms.	Kaushalya	Rajapakse	KK.
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				

This part is to be filled by the Topic Screening Staff members.

- a) Does the chosen research topic possess a comprehensive scope suitable for a final-year project?

Yes		No	
-----	--	----	--

- b) Does the proposed topic exhibit novelty?

Yes		No	
-----	--	----	--

- c) Do you believe they have the capability to successfully execute the proposed project?

Yes		No	
-----	--	----	--

- d) Do the proposed sub-objectives reflect the students' areas of specialization?

Yes		No	
-----	--	----	--

- e) Supervisor's Evaluation and Recommendation for the Research topic:

Acceptable: Mark/Select as necessary

Topic Assessment Accepted	
Topic Assessment Accepted with minor changes*	
Topic Assessment to be Resubmitted with major changes*	
Topic Assessment Rejected. Topic must be changed	

* Detailed comments given below

Comments

Staff Member's Name	Signature

***Important:**

1. According to the comments given by the evaluator, make the necessary modifications and get the approval by the **Evaluator**.
2. If the project topic is rejected, identify a new topic, and request the RP Team for a new topic assessment.